

Aquí tienes la **Explicación Técnica**. Piensa en esto como tu "acordeón" mental. Los profesores suelen hacer preguntas específicas para ver si entiendes la *lógica* detrás del código, no solo que funcione.

Aquí están los 4 conceptos clave que mueven tu proyecto:

1. TF-IDF (Term Frequency - Inverse Document Frequency)

- **¿Qué es?** Es una técnica estadística para evaluar qué tan importante es una palabra para un documento dentro de una colección.
- **¿Cómo se lo explicas al profe?**
 - "No podíamos usar un simple conteo de palabras, porque palabras como 'el', 'la' o 'y' aparecerían miles de veces pero no dicen nada sobre el libro."
 - "TF-IDF baja el puntaje de las palabras comunes (como 'el') y sube el puntaje de las palabras raras y únicas de ese libro (como 'Sherlock' o 'Vampiro')."
- **La Fórmula Mágica:**
 - **TF:** Frecuencia en el libro (¿Cuánto aparece aquí?).
 - **IDF:** Frecuencia inversa en todos los libros (¿Qué tan rara es en general?).

2. Vectorización (CountVectorizer)

- **¿Qué hace el código?** Convierte el texto humano en números.
- **Explicación:** "La computadora no sabe leer inglés ni español, solo entiende números. `CountVectorizer` convierte cada libro en una lista gigante de números (un vector) basada en nuestro vocabulario. Si el vocabulario tiene 10,000 palabras, cada libro es un vector de 10,000 posiciones."

3. Similitud del Coseno (Cosine Similarity)

- **¿Qué es?** Es la matemática que usamos para decir "este libro se parece a aquel".
- **La Analogía Visual:**
 - Imaginen que cada libro es una flecha en un gráfico.
 - Si dos libros tienen palabras muy parecidas, sus flechas apuntan casi a la misma dirección (el ángulo es pequeño).
 - Si son muy diferentes, las flechas apuntan a lados distintos (ángulo de 90 grados).
 - **Tu código calcula ese ángulo.** Si es cercano a 1, son idénticos; si es 0, no tienen nada que ver.

4. ¿Por qué PySpark? (Big Data)

- **La pregunta trampa:** "¿Por qué usar Spark para solo 100 libros si Python normal podría hacerlo?"
- **Tu respuesta ganadora:** "Para este prototipo usamos 100 libros, pero la arquitectura está diseñada para ser **escalable**. Si mañana quisiéramos procesar la biblioteca entera de Gutenberg (60,000+ libros), mi código en

PySpark podría distribuirse en un clúster de servidores sin cambiar la lógica, mientras que un script normal de Python se bloquearía por falta de memoria RAM."